

```
In [1]: import os
import torch
from torch import nn
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
```

```
In [2]: # 获取CUDA GPU设备信息
device = torch.accelerator.current_accelerator().type if torch.accelerator.is_avail
print(f"Using {device} device")
```

Using cuda device

```
In [3]: # 构建神经网络类
class NeuralNetwork(nn.Module):
    def __init__(self):
        super().__init__()
        self.flatten = nn.Flatten()
        self.linear_relu_stack = nn.Sequential(
            nn.Linear(28*28, 512),
            nn.ReLU(),
            nn.Linear(512, 512),
            nn.ReLU(),
            nn.Linear(512, 10),
        )

    def forward(self, x):
        x = self.flatten(x)
        logits = self.linear_relu_stack(x)
        return logits
```

```
In [4]: # 建立神经网络类的示例，并加载到GPU上
model = NeuralNetwork().to(device)
print(model)
```

```
NeuralNetwork(
  (flatten): Flatten(start_dim=1, end_dim=-1)
  (linear_relu_stack): Sequential(
    (0): Linear(in_features=784, out_features=512, bias=True)
    (1): ReLU()
    (2): Linear(in_features=512, out_features=512, bias=True)
    (3): ReLU()
    (4): Linear(in_features=512, out_features=10, bias=True)
  )
)
```

```
In [5]: # 随机生成一个28*28的灰度图像的张量
X = torch.rand(1, 28, 28, device=device)
```

```
In [6]: print(X)
```

```
tensor([[[[2.7763e-01, 4.2385e-01, 6.0669e-01, 3.6578e-01, 3.4049e-01,
2.8145e-01, 9.1923e-01, 8.8458e-02, 3.1069e-01, 5.9648e-01,
9.7900e-01, 3.7394e-01, 3.5058e-01, 2.9360e-02, 5.1220e-01,
3.8000e-01, 8.8967e-01, 5.4925e-01, 7.6982e-01, 1.5826e-01,
5.2243e-01, 7.0093e-01, 5.6416e-01, 6.9323e-01, 5.5412e-01,
5.5494e-01, 9.0393e-01, 3.5658e-01],
[1.2785e-01, 1.6277e-01, 8.9433e-01, 4.4581e-01, 9.3551e-01,
9.6028e-01, 9.4930e-01, 5.8508e-02, 5.1751e-01, 8.0962e-02,
5.5864e-01, 9.8877e-01, 3.0111e-01, 8.4304e-01, 4.2482e-01,
5.4063e-01, 2.6907e-01, 3.4310e-02, 1.6375e-01, 7.4713e-01,
2.9609e-01, 5.7959e-01, 1.1034e-03, 6.2581e-01, 5.5301e-01,
6.4846e-02, 5.1490e-01, 3.4058e-01],
[7.7697e-01, 2.8866e-01, 9.8256e-01, 3.9794e-01, 1.2495e-01,
9.4669e-01, 2.5477e-01, 3.7820e-01, 2.8856e-01, 2.1911e-01,
2.1646e-01, 3.0221e-01, 6.8459e-01, 3.7627e-01, 2.6186e-01,
5.1390e-02, 8.2376e-01, 5.2305e-01, 3.9413e-01, 5.7362e-01,
1.9130e-01, 2.0941e-01, 9.4009e-01, 3.5100e-01, 6.5769e-02,
7.8741e-01, 8.5753e-01, 8.9732e-02],
[3.3542e-02, 5.2321e-01, 2.6146e-01, 4.9234e-01, 7.7187e-01,
4.0928e-01, 1.3514e-01, 3.7213e-01, 9.6225e-01, 9.2289e-01,
5.1186e-01, 3.4071e-01, 9.5570e-01, 6.1446e-01, 2.3905e-03,
2.6533e-01, 4.5913e-01, 8.2388e-01, 3.1701e-01, 6.3964e-01,
2.0234e-01, 5.3277e-01, 4.5147e-01, 1.9201e-01, 3.6246e-01,
3.4684e-01, 1.3219e-01, 3.8832e-01],
[7.2470e-01, 5.5078e-01, 9.2749e-01, 8.8883e-01, 8.6049e-01,
7.2132e-01, 9.9795e-01, 8.1700e-01, 5.4836e-01, 6.9710e-02,
4.5534e-02, 7.7784e-02, 9.6889e-01, 1.6993e-02, 9.6723e-01,
3.0184e-01, 2.1572e-01, 1.7072e-01, 1.6427e-01, 7.8852e-01,
6.2955e-01, 3.8254e-01, 1.3708e-01, 1.5270e-01, 8.2680e-01,
5.9892e-02, 6.2022e-01, 4.1666e-01],
[9.9084e-02, 8.0752e-01, 1.7291e-01, 1.4777e-01, 3.8387e-01,
3.6507e-01, 3.6610e-01, 5.6156e-01, 1.6263e-01, 5.1635e-01,
4.8533e-01, 7.2108e-01, 3.0253e-01, 4.2003e-01, 4.4696e-01,
8.0377e-01, 6.9685e-01, 8.8200e-01, 5.9392e-01, 9.1979e-01,
4.8828e-01, 4.6831e-01, 8.6554e-04, 6.3579e-01, 1.5082e-01,
1.8213e-01, 8.1328e-01, 3.7987e-01],
[2.0079e-02, 7.2525e-01, 2.0261e-01, 9.6868e-01, 6.0504e-01,
1.8068e-01, 1.8520e-01, 4.1432e-01, 5.9228e-02, 9.7751e-01,
7.3726e-01, 6.9145e-01, 7.8309e-01, 3.2880e-01, 5.7951e-01,
2.3469e-01, 2.0090e-01, 4.7194e-01, 6.9211e-01, 5.3348e-01,
3.7669e-01, 6.3902e-01, 2.8054e-01, 4.7415e-01, 7.8603e-02,
9.5898e-01, 6.1489e-01, 1.4460e-01],
[4.8504e-01, 9.8462e-01, 2.8348e-01, 3.0209e-02, 1.5896e-01,
2.1187e-01, 6.4260e-01, 3.1577e-01, 3.0565e-01, 9.7805e-01,
7.1533e-01, 1.0008e-01, 8.2743e-02, 7.0923e-01, 9.6849e-01,
2.2291e-01, 8.1835e-01, 6.7067e-02, 3.3124e-01, 2.6443e-01,
1.1683e-01, 3.0635e-01, 6.9189e-01, 2.3449e-01, 2.6961e-01,
7.7274e-02, 9.8926e-01, 7.3924e-01],
[3.0244e-01, 5.6726e-02, 1.0170e-01, 5.4974e-02, 3.1983e-01,
8.4868e-01, 9.0171e-01, 2.3361e-01, 9.8099e-01, 9.5309e-01,
1.6841e-01, 1.2384e-01, 4.6858e-01, 1.6495e-01, 1.0772e-01,
2.0681e-01, 4.2570e-01, 4.5768e-01, 8.0435e-01, 8.6841e-01,
4.8776e-01, 9.2875e-01, 8.9583e-01, 2.6828e-01, 2.2367e-01,
7.3885e-01, 5.9430e-01, 5.0006e-01],
[6.9756e-01, 5.1112e-01, 9.7167e-01, 8.5352e-01, 8.3435e-01,
6.5652e-01, 1.2262e-01, 4.1939e-01, 2.1212e-01, 8.4359e-01,
```

3.3098e-01, 8.6498e-01, 1.1134e-01, 7.1118e-01, 5.4359e-02,
6.6280e-01, 2.5615e-01, 8.5621e-01, 8.2817e-01, 5.3556e-01,
7.6504e-01, 9.9109e-01, 6.4545e-01, 7.5154e-01, 9.9458e-01,
8.3887e-01, 4.9514e-01, 1.2027e-01],
[7.2339e-01, 8.1829e-01, 8.0194e-01, 4.0493e-01, 7.9542e-01,
2.7488e-01, 1.1318e-01, 7.1100e-01, 1.8564e-01, 1.2735e-01,
4.4248e-01, 3.2515e-01, 1.5310e-02, 6.9256e-01, 6.1726e-01,
9.8704e-01, 1.1490e-01, 5.2189e-01, 4.7075e-01, 9.5203e-01,
7.6756e-01, 7.1464e-01, 2.3978e-01, 1.5429e-01, 1.7221e-01,
8.8882e-01, 7.9583e-01, 3.1603e-01],
[8.4052e-01, 9.3135e-01, 4.2046e-01, 9.7289e-01, 7.5982e-01,
1.2131e-01, 8.5946e-01, 8.6212e-01, 4.1985e-01, 7.2731e-01,
1.2525e-01, 7.0076e-02, 5.5752e-01, 6.7083e-01, 5.2528e-01,
8.1301e-01, 1.4350e-01, 9.5976e-01, 2.4276e-01, 9.7146e-01,
2.1218e-01, 5.0739e-01, 2.3913e-01, 3.3257e-01, 6.8287e-01,
4.4810e-01, 9.9224e-01, 4.9256e-01],
[8.1896e-01, 7.0236e-01, 7.3208e-02, 7.1857e-01, 6.0174e-03,
5.0687e-02, 7.6900e-01, 1.2814e-01, 9.2347e-01, 9.2809e-01,
2.9056e-01, 3.2661e-02, 3.7917e-01, 2.6706e-01, 1.0098e-01,
1.4983e-01, 7.2071e-01, 5.1980e-01, 4.6999e-01, 4.2309e-01,
2.1373e-01, 9.9163e-01, 6.6304e-02, 9.3987e-01, 6.2108e-01,
9.6884e-01, 4.3548e-01, 4.8117e-01],
[9.0395e-01, 2.7591e-01, 7.6287e-01, 5.6157e-01, 4.8206e-03,
4.0110e-01, 3.1993e-01, 1.0679e-01, 2.5598e-01, 3.0644e-01,
7.0747e-01, 2.3145e-01, 4.7375e-02, 5.4498e-01, 8.2219e-01,
1.7444e-01, 9.6083e-01, 7.7391e-01, 1.7479e-02, 7.5026e-01,
7.6035e-01, 6.7747e-01, 8.2838e-01, 2.2374e-01, 7.9648e-02,
5.3817e-01, 8.6716e-01, 1.1383e-02],
[9.1326e-01, 9.0716e-01, 5.4487e-01, 7.4115e-01, 7.0735e-01,
7.7053e-01, 8.7164e-01, 2.1814e-01, 1.4107e-01, 2.6969e-02,
7.5684e-01, 2.1987e-01, 6.4526e-01, 7.3037e-01, 8.5300e-02,
9.4012e-01, 8.3689e-01, 1.2003e-01, 9.9317e-01, 1.4917e-01,
9.6770e-01, 1.4996e-01, 8.6379e-01, 1.7294e-01, 4.4583e-01,
7.4472e-01, 7.0183e-01, 3.1653e-01],
[6.3989e-01, 3.9593e-01, 3.4344e-01, 2.6437e-01, 5.1008e-01,
5.1194e-01, 2.5878e-01, 8.3131e-02, 9.2221e-01, 2.1364e-01,
8.8704e-01, 8.6774e-01, 2.4086e-01, 1.9405e-02, 5.1104e-01,
2.5169e-02, 1.9546e-02, 7.0959e-01, 9.6301e-01, 4.8892e-01,
1.1053e-01, 6.6511e-01, 6.6833e-01, 5.1794e-01, 2.5700e-01,
3.8041e-01, 9.8155e-01, 8.0596e-01],
[7.0144e-01, 8.3481e-01, 8.7974e-01, 3.2936e-01, 7.4061e-01,
9.1758e-02, 7.5503e-01, 1.0724e-01, 9.2530e-01, 9.2907e-01,
1.3369e-01, 6.6509e-01, 5.2496e-01, 9.9115e-02, 2.6903e-02,
6.7067e-01, 3.1450e-01, 3.8823e-01, 2.8881e-02, 7.1973e-02,
1.2044e-02, 7.4161e-02, 5.6974e-01, 6.6633e-01, 7.7255e-01,
2.8495e-01, 8.8998e-01, 7.7676e-01],
[4.7158e-01, 7.0607e-01, 3.2903e-01, 2.8886e-02, 3.8414e-01,
1.7300e-01, 2.3313e-01, 7.6532e-01, 6.9283e-01, 7.1154e-01,
4.6426e-01, 6.5171e-01, 3.0726e-01, 3.9739e-01, 5.2641e-01,
6.3987e-01, 9.0036e-02, 3.4537e-01, 3.2066e-01, 9.4843e-02,
1.1291e-01, 3.8893e-01, 4.2079e-01, 1.3194e-01, 1.5989e-01,
9.1726e-01, 8.7688e-02, 1.3353e-01],
[5.0360e-01, 1.5993e-01, 1.1745e-01, 3.1059e-01, 1.8072e-01,
9.9769e-01, 2.3751e-01, 6.3281e-01, 4.7323e-01, 5.3023e-01,
7.6356e-01, 9.8698e-01, 6.6730e-01, 9.7601e-01, 4.3161e-01,
5.4809e-01, 4.8522e-01, 7.9744e-01, 5.7815e-01, 3.9877e-02,

6.4021e-02, 3.2831e-01, 6.8939e-01, 6.7733e-01, 6.9271e-01,
8.2922e-01, 3.9590e-01, 1.5342e-01],
[9.9240e-01, 5.7064e-01, 5.4659e-01, 3.1224e-01, 6.3037e-01,
8.0658e-01, 5.9912e-01, 2.7507e-02, 8.2613e-01, 3.6577e-03,
7.0479e-02, 9.8179e-01, 2.2686e-01, 8.5958e-01, 9.1904e-02,
4.8919e-01, 7.0727e-02, 4.0475e-01, 2.1391e-01, 5.3152e-01,
5.9806e-01, 2.0393e-01, 8.2688e-01, 7.3921e-01, 5.3809e-01,
7.9918e-01, 8.4343e-01, 5.4781e-01],
[3.9528e-02, 5.1853e-01, 8.7023e-01, 5.5417e-02, 3.8511e-01,
7.3135e-01, 2.0800e-01, 9.3128e-01, 1.4882e-01, 6.1740e-01,
6.7964e-01, 2.2586e-01, 8.4361e-01, 5.7486e-01, 8.6554e-01,
7.2496e-01, 3.7001e-01, 9.9966e-01, 4.6872e-01, 4.2290e-01,
5.9772e-01, 8.8179e-01, 4.1632e-01, 7.3588e-01, 6.4834e-01,
3.3187e-01, 9.4101e-01, 8.4974e-01],
[6.6161e-01, 8.1243e-01, 2.7782e-01, 1.6102e-01, 3.2529e-01,
3.6850e-01, 5.5514e-01, 5.8242e-01, 9.2452e-01, 8.0957e-01,
6.7665e-01, 5.8990e-01, 1.4636e-02, 4.0357e-01, 6.1561e-01,
7.3237e-01, 7.7301e-01, 7.6846e-01, 1.8197e-01, 6.5093e-01,
9.7920e-01, 4.8468e-01, 9.5489e-01, 8.4875e-01, 9.1214e-01,
3.5498e-02, 3.7715e-02, 9.5127e-01],
[2.7998e-02, 8.8261e-01, 3.0338e-01, 5.0183e-01, 1.8102e-01,
7.9428e-01, 7.5462e-02, 4.5640e-01, 2.0248e-01, 7.8296e-01,
2.9258e-01, 5.4544e-01, 4.1573e-01, 4.3672e-01, 7.0183e-02,
4.1966e-01, 7.0610e-01, 9.1510e-01, 8.2712e-01, 6.2570e-01,
8.3053e-01, 5.9655e-01, 4.2487e-01, 9.4470e-01, 3.8756e-01,
6.7377e-02, 5.1639e-01, 8.0328e-01],
[6.7756e-01, 9.2482e-01, 1.8402e-01, 8.8392e-03, 8.0068e-01,
2.4395e-01, 4.0796e-01, 1.1461e-01, 8.1878e-01, 1.4140e-01,
8.3232e-01, 6.4395e-01, 5.4210e-01, 8.1529e-01, 5.7498e-01,
8.5417e-02, 3.3302e-01, 8.6050e-01, 7.5332e-01, 8.5305e-01,
9.8809e-01, 2.2209e-01, 9.5179e-01, 7.3323e-01, 5.7881e-01,
8.8022e-01, 9.0356e-01, 2.0405e-01],
[3.3238e-01, 3.2749e-02, 7.5552e-01, 4.4678e-01, 9.3459e-01,
5.5632e-01, 5.1515e-01, 4.9851e-01, 3.1996e-01, 5.3899e-01,
8.9697e-02, 3.3300e-01, 6.2129e-01, 2.2422e-02, 1.9413e-01,
5.5239e-01, 1.6948e-01, 5.2925e-01, 2.0590e-01, 3.8894e-01,
5.2258e-01, 7.8822e-01, 8.8816e-01, 8.4927e-01, 4.8617e-01,
4.8768e-01, 6.5486e-01, 3.1374e-01],
[1.1385e-01, 9.0672e-01, 5.7036e-01, 9.6211e-02, 2.8775e-01,
6.7537e-01, 2.9373e-01, 2.1285e-01, 2.2864e-01, 4.5935e-01,
9.9825e-01, 5.2475e-01, 8.7353e-02, 2.1612e-01, 5.9269e-01,
4.5674e-01, 4.7837e-01, 2.4103e-02, 1.4808e-01, 9.0965e-01,
5.2011e-01, 6.3152e-01, 4.1224e-01, 2.5513e-01, 9.6049e-01,
1.3330e-01, 6.9035e-01, 9.2335e-01],
[9.9431e-01, 2.2448e-01, 8.2714e-01, 6.4512e-01, 2.8385e-01,
4.6973e-01, 6.5807e-01, 6.6509e-01, 3.0088e-01, 4.0987e-01,
7.2637e-01, 5.3734e-01, 6.8788e-01, 9.7028e-01, 1.5502e-01,
6.2890e-01, 1.7410e-01, 8.9499e-01, 6.5878e-01, 4.4586e-01,
6.1259e-01, 6.4908e-01, 3.8064e-01, 2.0104e-02, 1.5479e-01,
4.1182e-01, 3.0423e-01, 7.8325e-01],
[7.0200e-01, 8.7213e-01, 7.6763e-01, 2.2265e-01, 2.6785e-01,
3.7697e-02, 1.7249e-01, 4.4461e-01, 6.6924e-01, 7.9417e-02,
4.1857e-01, 9.5202e-01, 7.9815e-01, 4.4006e-02, 9.3259e-02,
3.1834e-01, 3.4754e-01, 6.6681e-02, 9.4390e-01, 3.3689e-01,
7.7068e-01, 4.2676e-02, 8.4793e-01, 1.5158e-01, 5.6712e-01,
1.8441e-01, 4.9971e-01, 3.0510e-01]]], device='cuda:0')

```
In [7]: # 将数据输入到建好的神经网络模型, 获得输出
logits = model(X)
```

```
In [8]: print(logits)

tensor([[ -0.0342,  0.0513,  0.0667,  0.0158,  0.0514,  0.0118,  0.0751,  0.0754,
          0.0137,  0.0176]], device='cuda:0', grad_fn=<AddmmBackward0>)
```

```
In [9]: # 输出的结果中因为含有负数, 所以使用Softmax函数进行转换
pred_probab = nn.Softmax(dim=1)(logits)
```

```
In [10]: print(pred_probab)

tensor([[0.0933, 0.1016, 0.1032, 0.0981, 0.1016, 0.0977, 0.1041, 0.1041, 0.0979,
          0.0983]], device='cuda:0', grad_fn=<SoftmaxBackward0>)
```

```
In [11]: # 输出预测结果中标量值最大的对应的index
y_pred = pred_probab.argmax(1)
```

```
In [12]: print(f"Predicted class: {y_pred}")

Predicted class: tensor([7], device='cuda:0')
```

```
In [13]: # 构建3个灰度图像, 28*28像素
input_image = torch.rand(3,28,28)
print(input_image.size())

torch.Size([3, 28, 28])
```

```
In [14]: print(input_image)

tensor([[[[0.9505, 0.7330, 0.9028, ..., 0.5518, 0.9420, 0.1375],
          [0.1865, 0.9298, 0.1628, ..., 0.8570, 0.2161, 0.9665],
          [0.0911, 0.5900, 0.2296, ..., 0.4670, 0.5888, 0.1703],
          ...,
          [0.0650, 0.2964, 0.2258, ..., 0.4766, 0.8713, 0.8460],
          [0.2737, 0.4313, 0.8959, ..., 0.3395, 0.0612, 0.8074],
          [0.8840, 0.6795, 0.7462, ..., 0.0214, 0.4775, 0.7399]],

        [[0.0510, 0.3042, 0.0174, ..., 0.0814, 0.3900, 0.4237],
          [0.0516, 0.2442, 0.8040, ..., 0.3240, 0.7763, 0.6560],
          [0.0727, 0.2208, 0.1727, ..., 0.7119, 0.7606, 0.3984],
          ...,
          [0.1420, 0.8339, 0.2058, ..., 0.2872, 0.5954, 0.1902],
          [0.0344, 0.0811, 0.7117, ..., 0.0418, 0.6482, 0.6142],
          [0.4098, 0.6934, 0.4440, ..., 0.3727, 0.4417, 0.0879]],

        [[0.7646, 0.5696, 0.8767, ..., 0.0752, 0.9741, 0.0778],
          [0.2376, 0.2476, 0.5916, ..., 0.4846, 0.6349, 0.5899],
          [0.1033, 0.8184, 0.2793, ..., 0.8689, 0.5304, 0.9028],
          ...,
          [0.6409, 0.1912, 0.6950, ..., 0.3196, 0.9259, 0.1590],
          [0.7516, 0.7945, 0.3989, ..., 0.4062, 0.5716, 0.4214],
          [0.0566, 0.2638, 0.8607, ..., 0.6175, 0.9395, 0.9979]]]])
```

```
In [15]: # 通过nn.Flatten层, 将tensor([3,28,28])转换为: tensor([3,784])
flatten = nn.Flatten()
```

```
flat_image = flatten(input_image)
print(flat_image.size())
```

```
torch.Size([3, 784])
```

```
In [16]: print(flat_image)
```

```
tensor([[0.9505, 0.7330, 0.9028, ..., 0.0214, 0.4775, 0.7399],
        [0.0510, 0.3042, 0.0174, ..., 0.3727, 0.4417, 0.0879],
        [0.7646, 0.5696, 0.8767, ..., 0.6175, 0.9395, 0.9979]])
```

```
In [17]: # 通过nn.Linear层对输入的数据进行线性变换
layer1 = nn.Linear(in_features=28*28, out_features=20)
hidden1 = layer1(flat_image)
print(hidden1.size())
```

```
torch.Size([3, 20])
```

```
In [18]: print(hidden1)
```

```
tensor([[ 0.1353, -0.1623,  0.1260,  0.8000, -0.4293, -0.5048, -0.0607, -0.2056,
         -0.3937, -0.1709, -0.1802, -0.6804, -0.4060,  0.1640, -0.0878, -0.2103,
          0.3428,  0.4806, -0.0100,  0.4306],
        [ 0.7443, -0.3252,  0.2317,  0.4928, -0.7591, -0.6449,  0.0836, -0.1884,
          0.0954, -0.1962, -0.0129, -0.5353, -0.9013,  0.4133, -0.1210, -0.2066,
          0.3878,  0.3876,  0.1471,  0.5384],
        [ 0.5812, -0.0778,  0.1967,  0.8081, -0.5683, -0.3658, -0.0516, -0.0489,
         -0.0566, -0.1965,  0.0874, -0.2688, -0.5498,  0.0410, -0.3652, -0.6512,
          0.2863,  0.0737,  0.1046,  0.2669]], grad_fn=<AddmmBackward0>)
```

```
In [19]: # ReLU激活函数, 引入非线性变换
print(f"Before ReLU: {hidden1}\n\n")
hidden1 = nn.ReLU()(hidden1)
print(f"After ReLU: {hidden1}")
```

```
Before ReLU: tensor([[ 0.1353, -0.1623,  0.1260,  0.8000, -0.4293, -0.5048, -0.0607,
         -0.2056,
         -0.3937, -0.1709, -0.1802, -0.6804, -0.4060,  0.1640, -0.0878, -0.2103,
          0.3428,  0.4806, -0.0100,  0.4306],
        [ 0.7443, -0.3252,  0.2317,  0.4928, -0.7591, -0.6449,  0.0836, -0.1884,
          0.0954, -0.1962, -0.0129, -0.5353, -0.9013,  0.4133, -0.1210, -0.2066,
          0.3878,  0.3876,  0.1471,  0.5384],
        [ 0.5812, -0.0778,  0.1967,  0.8081, -0.5683, -0.3658, -0.0516, -0.0489,
         -0.0566, -0.1965,  0.0874, -0.2688, -0.5498,  0.0410, -0.3652, -0.6512,
          0.2863,  0.0737,  0.1046,  0.2669]], grad_fn=<AddmmBackward0>)
```

```
After ReLU: tensor([[0.1353, 0.0000, 0.1260, 0.8000, 0.0000, 0.0000, 0.0000, 0.0000,
         0.0000,
         0.0000, 0.0000, 0.0000, 0.1640, 0.0000, 0.0000, 0.3428, 0.4806,
          0.0000, 0.4306],
        [0.7443, 0.0000, 0.2317, 0.4928, 0.0000, 0.0000, 0.0836, 0.0000, 0.0954,
          0.0000, 0.0000, 0.0000, 0.0000, 0.4133, 0.0000, 0.0000, 0.3878, 0.3876,
          0.1471, 0.5384],
        [0.5812, 0.0000, 0.1967, 0.8081, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0874, 0.0000, 0.0000, 0.0410, 0.0000, 0.0000, 0.2863, 0.0737,
          0.1046, 0.2669]], grad_fn=<ReluBackward0>)
```

```
In [20]: # nn.Sequential 容器模块
seq_modules = nn.Sequential(
    flatten,
    layer1,
    nn.ReLU(),
    nn.Linear(20, 10)
)
input_image = torch.rand(3,28,28)
logits = seq_modules(input_image)
```

```
In [21]: print(logits)

tensor([[ 0.0852, -0.2434,  0.0790, -0.1398,  0.1236,  0.0659, -0.1486,  0.0805,
          -0.2511, -0.2197],
        [ 0.0360, -0.3459,  0.0378, -0.1486,  0.0640,  0.0812, -0.0915,  0.1584,
          0.0118, -0.3658],
        [ 0.1616, -0.2036,  0.0170, -0.0219, -0.0191,  0.1071, -0.1173,  0.0811,
          -0.1755, -0.2311]], grad_fn=<AddmmBackward0>)
```

```
In [22]: # 对预测结果进行变换
softmax = nn.Softmax(dim=1)
pred_probab = softmax(logits)
```

```
In [23]: print(pred_probab)

tensor([[0.1140, 0.0821, 0.1133, 0.0910, 0.1185, 0.1118, 0.0903, 0.1135, 0.0815,
          0.0841],
        [0.1082, 0.0738, 0.1083, 0.0899, 0.1112, 0.1132, 0.0952, 0.1222, 0.1056,
          0.0724],
        [0.1213, 0.0842, 0.1050, 0.1010, 0.1013, 0.1149, 0.0918, 0.1119, 0.0866,
          0.0819]], grad_fn=<SoftmaxBackward0>)
```

```
In [24]: # 查看神经网络模型的参数情况:

print(f"Model structure: {model}\n\n")

for name, param in model.named_parameters():
    print(f"Layer: {name} | Size: {param.size()} | Values : {param[:2]} \n")
```

```

Model structure: NeuralNetwork(
  (flatten): Flatten(start_dim=1, end_dim=-1)
  (linear_relu_stack): Sequential(
    (0): Linear(in_features=784, out_features=512, bias=True)
    (1): ReLU()
    (2): Linear(in_features=512, out_features=512, bias=True)
    (3): ReLU()
    (4): Linear(in_features=512, out_features=10, bias=True)
  )
)

```

```

Layer: linear_relu_stack.0.weight | Size: torch.Size([512, 784]) | Values : tensor
([[-0.0305,  0.0348, -0.0304, ...,  0.0047, -0.0280, -0.0002],
 [ 0.0128,  0.0259,  0.0059, ..., -0.0082,  0.0033, -0.0290]],
 device='cuda:0', grad_fn=<SliceBackward0>)

```

```

Layer: linear_relu_stack.0.bias | Size: torch.Size([512]) | Values : tensor([ 0.000
7, -0.0040], device='cuda:0', grad_fn=<SliceBackward0>)

```

```

Layer: linear_relu_stack.2.weight | Size: torch.Size([512, 512]) | Values : tensor
([[-0.0019,  0.0184,  0.0200, ...,  0.0137,  0.0059,  0.0326],
 [ 0.0086,  0.0364,  0.0333, ..., -0.0393, -0.0192, -0.0016]],
 device='cuda:0', grad_fn=<SliceBackward0>)

```

```

Layer: linear_relu_stack.2.bias | Size: torch.Size([512]) | Values : tensor([-0.024
9, -0.0192], device='cuda:0', grad_fn=<SliceBackward0>)

```

```

Layer: linear_relu_stack.4.weight | Size: torch.Size([10, 512]) | Values : tensor
([[-0.0027,  0.0036, -0.0151, ...,  0.0096,  0.0293,  0.0188],
 [ 0.0322, -0.0026, -0.0100, ...,  0.0410,  0.0025, -0.0124]],
 device='cuda:0', grad_fn=<SliceBackward0>)

```

```

Layer: linear_relu_stack.4.bias | Size: torch.Size([10]) | Values : tensor([-0.0270,
 0.0363], device='cuda:0', grad_fn=<SliceBackward0>)

```

```

In [25]: # Pytorch的自动微分机制torch.autograd

x = torch.ones(5) # input tensor
y = torch.zeros(3) # expected output
w = torch.randn(5, 3, requires_grad=True)
b = torch.randn(3, requires_grad=True)
z = torch.matmul(x, w)+b
loss = torch.nn.functional.binary_cross_entropy_with_logits(z, y)

```

```

In [26]: x

```

```

Out[26]: tensor([1., 1., 1., 1., 1.])

```

```

In [27]: w

```



```
Out[27]: tensor([[ -0.2363, -1.1432,  0.3804],
                 [-0.8250,  0.2017,  0.3847],
                 [ 0.3579, -0.6434, -0.7507],
                 [-1.1131,  0.9815,  0.3064],
                 [ 0.0466,  0.7115, -0.8278]], requires_grad=True)
```

```
In [28]: y
```

```
Out[28]: tensor([0., 0., 0.])
```

```
In [29]: # 计算参数的梯度 gradient
loss.backward(retain_graph=True) # 加上retain_graph=True就可以多次计算参数梯度，如果没
print(w.grad)
print(b.grad)
```

```
tensor([[0.0802, 0.2702, 0.0644],
        [0.0802, 0.2702, 0.0644],
        [0.0802, 0.2702, 0.0644],
        [0.0802, 0.2702, 0.0644],
        [0.0802, 0.2702, 0.0644]])
tensor([0.0802, 0.2702, 0.0644])
```

```
In [30]: # 计算参数的梯度 gradient
loss.backward(retain_graph=False)
print(w.grad)
print(b.grad)
```

```
tensor([[0.1605, 0.5404, 0.1288],
        [0.1605, 0.5404, 0.1288],
        [0.1605, 0.5404, 0.1288],
        [0.1605, 0.5404, 0.1288],
        [0.1605, 0.5404, 0.1288]])
tensor([0.1605, 0.5404, 0.1288])
```